Matplotlib for AI Engineering

Detailed practical guide with examples for data visualization, EDA, and ML model analysis

Goal: Learn the Matplotlib skills needed to visualize datasets, understand distributions and relationships, debug ML models, and communicate training and evaluation results.

Target: practical proficiency in 3–5 focused days. Advanced Matplotlib internals are not required for your AI Engineering foundation.

1. Setup and Basic Workflow

Install and import Matplotlib:

```
pip install matplotlib

import matplotlib.pyplot as plt
```

Basic workflow:

```
fig, ax = plt.subplots()
ax.plot(x, y)
plt.show()
```

Figure is the overall canvas. **Axes** is an individual plotting area. Learn the object-oriented style using **fig** and **ax** because it scales better to complex plots.

2. Figure and Axes

```
fig, ax = plt.subplots()

ax.plot([1, 2, 3], [10, 20, 15])
ax.set_title("Example")
ax.set_xlabel("X")
ax.set_ylabel("Y")

plt.show()
```

Mental model: Figure → Axes → plot, title, labels, legend, grid. One Figure can contain multiple Axes.

3. Line Plots

Line plots show trends and are heavily used for time series and ML training curves.

```
x = [1, 2, 3, 4, 5]
y = [10, 15, 13, 20, 25]

fig, ax = plt.subplots()
ax.plot(x, y)
ax.set_title("Sales Over Time")
ax.set_xlabel("Day")
ax.set_ylabel("Sales")
plt.show()
```

Common uses: trends, time series, loss, accuracy, latency, system metrics.

4. Multiple Lines and Legends

```
epochs = [1, 2, 3, 4, 5]
train_loss = [1.2, .9, .7, .5, .4]
val_loss = [1.3, 1.0, .8, .7, .75]

fig, ax = plt.subplots()
ax.plot(epochs, train_loss, label="Training")
ax.plot(epochs, val_loss, label="Validation")
ax.set_xlabel("Epoch")
ax.set_ylabel("Loss")
```

```
ax.legend()
plt.show()
```

Use **label=** with each line and **ax.legend()** to explain the lines.

5. Bar Charts

```
categories = ["Python", "Java", "C++"]
values = [80, 65, 50]

fig, ax = plt.subplots()
ax.bar(categories, values)
ax.set_title("Skill Comparison")
ax.set_ylabel("Score")
plt.show()

# Horizontal:
ax.barh(categories, values)
```

Use bars for discrete category comparisons, counts, and metric comparisons.

6. Scatter Plots

```
x = [1, 2, 3, 4, 5]
y = [2, 4, 3, 8, 10]

fig, ax = plt.subplots()
ax.scatter(x, y)
ax.set_xlabel("Feature")
ax.set_ylabel("Target")
ax.set_title("Feature vs Target")
plt.show()
```

Scatter plots are essential for EDA and ML: feature vs target, age vs salary, actual vs predicted, and other relationships.

7. Histograms

```
import numpy as np

data = np.random.randn(1000)

fig, ax = plt.subplots()
ax.hist(data, bins=30)
ax.set_title("Distribution")
ax.set_xlabel("Value")
ax.set_ylabel("Frequency")
plt.show()
```

Histograms help inspect distribution, range, concentration, skewness, and possible unusual values. **bins** controls grouping.

8. Box Plots

```
data = [10, 12, 13, 15, 16, 17, 100]

fig, ax = plt.subplots()
ax.boxplot(data)
ax.set_title("Box Plot")
plt.show()
```

Box plots summarize spread and help identify potential outliers.

9. Titles, Labels, Legends and Grid

```
ax.set_title("Model Performance")
ax.set_xlabel("Epoch")
ax.set_ylabel("Accuracy")
ax.legend()
```

```
ax.grid(True)
```

A good chart should make its meaning clear without requiring the reader to guess what the axes represent.

10. Subplots

```
fig, axes = plt.subplots(2, 2)

axes[0, 0].plot(x, y)
axes[0, 1].bar(categories, values)
axes[1, 0].scatter(x, y)
axes[1, 1].hist(data)

plt.tight_layout()
plt.show()
```

Subplots are useful for EDA reports, comparing experiments, and showing related metrics in one figure.

11. Figure Size and Layout

```
fig, ax = plt.subplots(figsize=(10, 6))

fig, axes = plt.subplots(
    2, 2,
    figsize=(10, 7)
)

plt.tight_layout()
```

Figure size is measured in inches. Layout tools help prevent labels and titles from overlapping.

12. Axis Limits and Ticks

```
ax.set_xlim(0, 100)
ax.set_ylim(0, 1)

ax.set_xticks([0, 10, 20, 30])
ax.set_yticks([0, 25, 50, 75, 100])
```

Learn basic limits and ticks first. Advanced formatters can wait until you need dates, percentages, currency, or logarithmic axes.

13. Saving Figures

```
fig.savefig(
    "training_curve.png",
    dpi=300,
    bbox_inches="tight"
)
```

Common formats: PNG, JPG, SVG, PDF. Saving plots is important for reports, experiment records, documentation, and presentations.

14. Basic Customization

```
ax.set_title(...)
ax.set_xlabel(...)
ax.set_ylabel(...)
ax.grid(True)
ax.legend()
ax.set_xlim(...)
ax.set_ylim(...)
```

Do not memorize hundreds of styling parameters. Learn the core controls and look up specialized options when a project requires them.

15. NumPy + Matplotlib

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 100)
y = np.sin(x)

fig, ax = plt.subplots()
ax.plot(x, y)
ax.set_title("Sine Wave")
plt.show()
```

Concept: **NumPy** → **numerical arrays** → **Matplotlib** → **visualization**.

16. Pandas + Matplotlib

```
df["salary"].plot(kind="hist")
plt.show()

df.plot(
    x="age",
    y="salary",
    kind="scatter"
)
plt.show()
```

Pandas can use Matplotlib as its visualization backend, making EDA convenient.

17. Training Curves

```
epochs = [1, 2, 3, 4, 5]
loss = [1.2, .9, .7, .5, .4]

fig, ax = plt.subplots()
ax.plot(epochs, loss)
ax.set_xlabel("Epoch")
ax.set_ylabel("Loss")
ax.set_title("Training Loss")
plt.show()
```

Training curves let you inspect how a model's loss changes during optimization.

18. Training vs Validation Curves

```
fig, ax = plt.subplots()

ax.plot(epochs, train_loss, label="Train")
ax.plot(epochs, val_loss, label="Validation")

ax.set_xlabel("Epoch")
ax.set_ylabel("Loss")
ax.legend()
plt.show()
```

Compare training and validation behavior to inspect generalization and patterns such as validation performance worsening while training performance continues improving.

19. Actual vs Predicted

```
fig, ax = plt.subplots()

ax.scatter(y_actual, y_predicted)
ax.set_xlabel("Actual")
ax.set_ylabel("Predicted")
ax.set_title("Actual vs Predicted")
plt.show()
```

For regression, this visualization complements metrics such as MAE, MSE, and R^2.

20. Residual Plots

```
residuals = y_actual - y_predicted

fig, ax = plt.subplots()
ax.scatter(y_predicted, residuals)
ax.axhline(0)
ax.set_xlabel("Predicted")
ax.set_ylabel("Residual")
plt.show()
```

Residual plots can reveal systematic errors, changing variance, or unusual observations.

21. Confusion Matrix

```
from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_predictions(
    y_test,
    y_pred
)
plt.show()
```

```
Predicted
              0          1

Actual 0      TN         FP
Actual 1      FN         TP
```

You do not need to manually draw the matrix. Learn to interpret TN, FP, FN, and TP and use Scikit-learn's display helper.

22. Correlation / Heatmap

```
correlation = np.array([
    [1.0, .8, .2],
    [.8, 1.0, .4],
    [.2, .4, 1.0]
])

fig, ax = plt.subplots()
image = ax.imshow(correlation)
fig.colorbar(image)
plt.show()
```

Matplotlib's **imshow()** can visualize matrices. In practical data science, Seaborn is often used for statistical heatmaps.

23. Choosing the Right Chart

Line: trends/time/training curves.
Bar: category comparison.
Scatter: relationships between numerical variables.
Histogram: numerical distributions.
Box plot: spread and potential outliers.
Actual vs predicted: regression inspection.
Confusion matrix: classification outcomes.
Heatmap: matrices and correlations.

24. What to Learn Later

Do not spend weeks on Matplotlib internals, custom backends, advanced transforms, complex animation, custom Artists, specialized projections, advanced rendering, or exhaustive styling APIs.

Learn advanced features when a real project needs them. For AI Engineering, your time is better spent on NumPy, Pandas, statistics, ML, deep learning, LLMs, RAG, agents, evaluation, APIs, and deployment.

25. Practice Project

Take one real dataset and create an EDA + ML visualization report containing:

1. Histograms for numerical features.
2. A box plot for outliers.
3. A bar chart for a categorical variable.
4. A scatter plot for an important feature/target relationship.
5. A correlation matrix visualization.
6. Training/validation curves if you train a model.
7. Actual-vs-predicted for regression OR a confusion matrix for classification.
8. Save important figures to disk.

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 2, figsize=(10, 8))

axes[0, 0].hist(df["age"], bins=20)
axes[0, 0].set_title("Age Distribution")

axes[0, 1].boxplot(df["income"].dropna())
axes[0, 1].set_title("Income")

axes[1, 0].scatter(df["age"], df["income"])
axes[1, 0].set_xlabel("Age")
axes[1, 0].set_ylabel("Income")

axes[1, 1].bar(["A", "B", "C"], [10, 20, 15])
axes[1, 1].set_title("Category Counts")

plt.tight_layout()
plt.savefig("eda_report.png", dpi=300, bbox_inches="tight")
plt.show()
```

26. Practice Exercises

1. Create a line plot for 10 days of sales.
2. Create a bar chart comparing five categories.
3. Create a scatter plot between age and salary.
4. Generate 1,000 random values and visualize their distribution.
5. Create a box plot and inspect unusual values.
6. Create a 2×2 subplot with four chart types.
7. Plot training and validation loss together.
8. Plot actual vs predicted values for regression.
9. Create a residual plot.
10. Display a confusion matrix with Scikit-learn.
11. Visualize a correlation matrix.
12. Build a complete EDA visualization report from a real dataset.

27. AI Engineer Checklist

Must master: Figure/Axes, line, bar, scatter, histogram, labels, titles, legends, grid, subplots, NumPy/Pandas integration, training curves, actual-vs-predicted, and basic model evaluation plots.

Should know: box plots, axis limits, ticks, figure sizing, saving figures, matrix visualization.

Learn later: advanced styling, animation, transforms, custom Artists, projections, and Matplotlib internals.

Final target: You should be able to take a dataset or model result and visually inspect distributions, relationships, outliers, trends, and model behavior without following a tutorial for every chart.

28. Recommended 5-Day Plan

Day 1: Figure, Axes, line, bar, scatter.
Day 2: Histogram, box plot, labels, titles, legends, grid.
Day 3: Subplots, figure size, limits, ticks, saving figures.
Day 4: NumPy + Pandas visualization and EDA.
Day 5: Training curves, actual vs predicted, residuals, confusion matrix, and mini project.

29. AI Engineering Roadmap

Python → NumPy → Pandas → Matplotlib/Seaborn → SQL → Statistics/Math → Scikit-learn → PyTorch → Deep Learning → Transformers/LLMs → Embeddings → RAG → Agents → Production AI Engineering

Matplotlib is a supporting tool. Once you can confidently visualize and debug data/model behavior, move your study time toward statistics, machine learning, deep learning, LLMs, RAG, agents, evaluation, APIs, deployment, and production AI systems.